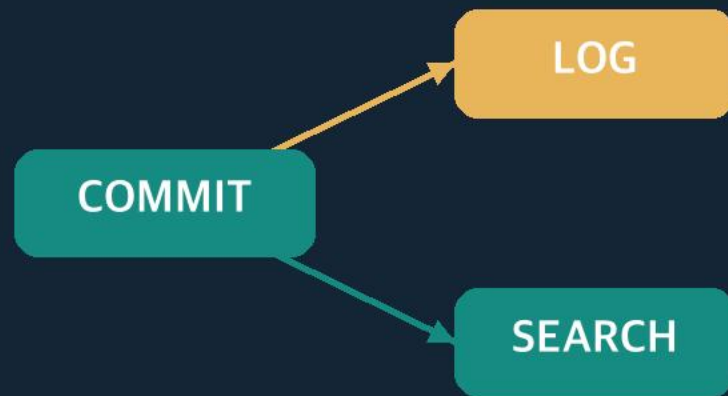


Mini Git

실행부터 평가 설명까지

직접 실행하고, 근거로 확인하고, 내 말로 설명하기



과제

B3-2 · 자료구조와 알고리즘

검증

Python 3.12.13 · 테스트 15개 통과

대상

처음 배우는 학습자 · 평가 발표 준비

상세 설명

이 자료는 AI-SW-Basic/b3-2의 실제 구현과 실행 기록을 바탕으로 만들었습니다. 순서는 실행, 검증, 원리 설명입니다. Mini Git은 파일 내용을 관리하는 완전한 Git이 아니라 커밋 메타데이터로 그래프·탐색·정렬·검색을 배우는 프로그램입니다.

발표에서 답할 세 가지

핵심 · 그림 · 실행

01 어떻게 실행했나요?

b3-2 폴더에서 `python main.py` 명령을 한 줄씩 입력합니다.

02 어떻게 검증하나요?

화면의 결과를 확인하고 자동 테스트를 실행합니다. 정상·오류·경계 입력을 비교합니다.

03 근거는 어디 있나요?

코드: `mini_git/` 테스트: `tests/` 기록: `제출/results/`

상세 설명 · 코드 · 문답 전체

이 자료는 `mini_git/`에 구현한 CLI 기반 Mini Git이 과제 목표를 어떻게 만족하는지, 평가자 앞에서 아주 쉬운 말로 설명할 수 있도록 정리한 문서입니다. 외워서 말하기보다, 아래 설명을 본인 말투로 바꿔서 연습하는 용도로 사용하세요. 검증 근거는 실행 결과 (제출/실행결과.md)와 회귀 테스트 (`tests/test_mini_git.py`)입니다. 2026-10-09, Python 3.12.13에서 회귀 테스트 15개를 통과했습니다.

프로그램은 실행된다는 사실만으로 정답이라고 할 수 없습니다. 과제의 조건을 결과와 연결해야 합니다. 예를 들어 LOG는 출력이 있는지뿐 아니라 부모가 먼저 나오는지 봐야 합니다. 오늘의 세 가지 질문은 실행 방법, 확인 기준, 증거 위치입니다.

명령 → 기대 결과 → 근거 파일을 연결해서 설명합니다.

근거: `README.md` · `제출/실행결과.md` · `평가질문_설명자료.md`

먼저 폴더의 역할을 구분하세요

AI-SW-Basic/b3-2/

main.py	실행 시작점
README.md	실행 설명
mini_git/	실제 구현
cli.py / repository.py	입력·상태 관리
traversal.py	그래프 탐색
sorting.py / index.py	정렬·검색
tests/test_mini_git.py	자동 검사
제출/실행결과.md	결과 안내
제출/results/	원본 결과

상세 설명

모든 파일 경로는 b3-2를 기준으로 말합니다. main.py만 전달하면 내부 패키지를 찾을 수 없으므로 mini_git 폴더도 함께 있어야 합니다. 과제 원문은 요구사항, 최종검수결과는 요구사항별 판단, 평가질문_설명자료는 답변 연습 자료입니다. results의 텍스트는 실행 흔적이며 테스트 소스와 구분합니다.

README는 사용 설명서, tests는 검사 도구, results는 검사 기록입니다.

실행은 이 세 줄로 시작합니다

터미널에 입력

```
cd ~/AI-SW-Basic/b3-2  
python3 --version  
python3 main.py
```

정상 화면

Mini Git CLI — 종료하려면 exit 또는 quit을 입력하세요.

```
mini-git>
```

알아두기

python 명령이 설정되어 있으면 python main.py도 같습니다.

추가 라이브러리 설치의 필요 없습니다.

종료: exit 또는 quit

상세 설명

터미널은 프로그램을 시작하는 장소입니다. 시작 뒤 mini-git>가 보이면 그때부터는 Mini Git 명령을 입력합니다. python과 python3는 컴퓨터 설정에 따라 이름이 다릅니다. 이번 검증에 사용한 버전은 Python 3.12.13입니다. 표시된 경로가 다르면 실제 프로젝트 위치로 cd 하세요.

Python 3.10 이상인지 확인하고, b3-2 안에서 실행합니다.

브랜치가 갈라지는 상황을 만들어 봅니다

mini-git> 뒤에 한 줄씩 입력

```
init "Alice Kim"  
commit "Initial commit"  
branch feature  
switch feature  
commit "Add login feature"  
switch main  
commit "Add payment feature"  
log
```

세 커밋의 관계

Initial: 부모 없음

Login: Initial이 부모

Payment: Initial이 부모

main → Payment

feature → Login

상세 설명

첫 커밋을 만든 다음 feature를 만듭니다. feature는 그 순간의 커밋을 가리키는 이름표입니다. feature에서 로그인 커밋을 만들고 main으로 돌아 오면 main은 여전히 Initial을 가리킵니다. 그래서 결제 커밋도 Initial을 부모로 합니다. branch는 전환하지 않으며 switch가 전환합니다. 사용자명·메시지에 공백이 있으면 따옴표를 씁니다.

COMMIT은 현재 브랜치의 끝을 부모로 삼고, 그 브랜치의 끝을 이동합니다.

출력에 무엇이 보이면 성공일까요?

명령 / 기대 결과 / 확인 기준

INIT / main·사용자 표시 / 초기 상태 생성

COMMIT / [branch hash] message / hash와 메시지 표시

LOG / 세 커밋 출력 / Initial이 두 자식보다 먼저

SEARCH login / 1개 / Login 메시지만 포함

SEARCH --author="Alice Kim" / 3개 / 작성자 정확히 일치

LOG --sort-by=date / 날짜 오름차순 / timestamp 비교

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습 후

LOG → SEARCH login → SEARCH --author="Alice Kim" → LOG --sort-by=date

확인: 5페이지의 새 실습 기준: 커밋 3개, login 검색 1개, 작성자 검색 3개, 날짜 오름차순

hash는 커밋의 식별자이지 외워야 할 정답이 아닙니다. 실제 화면에 나온 hash를 사용해야 합니다. 로그에서는 hash, author, timestamp, message가 식별 가능해야 합니다. 한 세션에서는 작성자가 한 명이므로 author 정렬의 순서 변화가 눈에 보이지 않을 수 있습니다. 서로 다른 작성자의 정렬은 테스트에서 별도로 확인합니다.

문구보다 동작의 규칙을 확인합니다. hash는 실행마다 달라질 수 있습니다.

PATH와 ANCESTORS는 실제 hash로 확인합니다

최단 경로 확인 — PATH

PATH 5fa901 837f82
기대 경로:
5fa901 → 0a7990 → 837f82
최단 거리 = 간선 2개

조상 확인 — ANCESTORS

ANCESTORS 837f82
기대 조상: 0a7990

ANCESTORS 0a7990
기대: No ancestors.

먼저 LOG에서 hash 확인

LOG
0a7990: 최초 커밋
5fa901: 로그인 커밋
837f82: 결제 커밋
세션이 바뀌면 실제 hash로 교체

상세 설명

mini-git> 프롬프트에서 LOG로 hash를 확인한 뒤 PATH 5fa901 837f82와 ANCESTORS 837f82를 입력합니다. 이 명령은 사용자가 제시한 실행 로그의 hash를 사용한 예시이며, 로그인과 결제 커밋이 최초 커밋을 공통 부모로 갖는 흐름을 기준으로 합니다. 새 세션에서는 hash가 달라지므로 현재 LOG의 값으로 바꿉니다. PATH는 Login → Initial → Payment로 간선 두 개를 지나며, 화살표는 경로 순서입니다. ANCESTORS는 부모 방향만 따라가고 자신은 제외합니다. ANCESTORS 0a7990은 No ancestors.를 출력합니다. PATH가 자기 자신이면 간선 수는 0입니다. 경로가 없으면 No path, 존재하지 않는 hash면 Unknown commit입니다.

PATH는 양방향으로 길을 찾고, ANCESTORS는 부모 방향만 따라갑니다.

자동 검증은 한 명령으로 재현합니다

b3-2 터미널에서 실행

```
python3 -B -m unittest discover -s tests -v
```

포함된 검사

무작위 DAG 50개 · 정렬 데이터 최대 1,000개
부모 5,000개 · hash 강제 충돌
실제 REPL 실행 · exit/quit 종료

상세 설명

unittest는 Python에 들어 있는 테스트 실행 도구입니다. -s tests는 tests 폴더를 검사하라는 뜻이고 -v는 검사 이름을 자세히 보여줍니다. -B는 파이썬 캐시 파일 생성을 막는 선택 옵션입니다. 각 test 함수는 여러 assert와 반복 입력을 포함합니다. FAIL 또는 ERROR가 나오면 해당 이름과 마지막 오류 메시지를 읽어야 합니다. tests의 코드는 검사 방법이며 results/11은 검사 기록입니다.

15

TESTS PASSED

Ran 15 tests
OK

15개 테스트는 여러 입력을 반복 확인합니다. 성공 표시는 OK입니다.

평가자에게 보여 줄 근거 위치

확인하고 싶은 것 / 파일

실행 안내 / README.md, 가이드.md

요구사항별 판단 / 최종검수결과.md

실행 결과 요약 / 제출/실행결과.md

기본 기능 원본 / 제출/results/01~09_*.txt

보완 동작 원본 / 제출/results/10_보완_회귀검증.txt

테스트 원본 결과 / 제출/results/11_자동검증.txt

검사 소스 / tests/test_mini_git.py

알고리즘 구현 / mini_git/traversal.py, sorting.py, index.py

상세 설명

01~09 및 10 중 09를 제외한 기록은 dispatch 함수에 실제 명령을 전달한 출력입니다. 09는 main.py 별도 프로세스 종료 기록입니다. 테스트 안에서도 main.py를 직접 실행합니다. 따라서 모든 텍스트를 화면 캡처라고 부르지 않습니다. 오늘 PPT와 발표 노트는 제출/발표자료에 저장합니다.

결과 → 검사 방법 → 구현 순서로 파일을 열면 설명하기 쉽습니다.

검수에서 발견한 문제를 이렇게 고쳤습니다

문제 → 수정 → 확인

반복 단어가 커밋 중복 출력 → 커밋별 토큰 중복 제거 → 1개만 반환
공백 포함 검색어 0건 → 연속 토큰 구문 검색 → login feature 검색
기존 BRANCH 덮어쓰기 → Invalid args로 거부 → 모든 HEAD 보존
조상 큐 pop(0) 비용 → 인덱스로 읽기 → 다중 부모 검증
공백뿐인 입력 수용 → .strip() 검사 → 표준 오류 반환
버전·hash 설명 부정확 → 실제 환경·충돌 재시도 설명 → 문서 갱신

실행 확인 · 명령은 한 줄씩 입력

일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k phrase_order
```

확인: 연속 구문·반복 토큰·대소문자·공백 처리 검증 · 마지막 OK 확인

회귀 테스트는 한 번 고친 문제가 다시 생기지 않게 하는 검사입니다. 중복 검색은 메시지 login login LOGIN으로 확인합니다. 검색 결과는 단어 등장 횟수가 아니라 커밋 개수입니다. branch main을 다른 브랜치에서 호출해도 main이 이동하지 않아야 합니다. Python 3.9.6의 과거 검수 대신 3.12.13 검증을 기준으로 설명합니다.

기존의 전체 통과 결론을 재검수하고, 오류를 고친 뒤 테스트를 추가했습니다.

입력과 알고리즘을 나눈 이유

핵심 · 그림 · 실행

입력·출력

main.py → cli.py 문자열 파싱·명령 선택·결과 출력

저장소 상태

repository.py 사용자·브랜치·HEAD·커밋 관리

알고리즘

traversal.py: 탐색 sorting.py: 정렬 index.py: 역색인

실행 확인 · 일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s
tests -v -k actual_entrpoint
```

확인: main.py 실행과 실제 CLI 입출력·종료 검증 · 마지막 OK 확인

상세 설명 · 코드 · 문답 전체

이번 과제는 그래프 전용 라이브러리와 sorted()/list.sort() 없이, 커밋 그래프 탐색(위상 정렬·최단 경로·조상)과 정렬 알고리즘 을 직접 구현해 만든 CLI 기반 Mini Git입니다.

main.py ← 엔트리 포인트

mini_git/

— cli.py	REPL, 명령어 파싱, 출력 포매팅
— repository.py	저장소 상태(브랜치/HEAD/사용자) 관리
— commit.py	Commit 노드 + 세션 내 유일한 hash 생성
— traversal.py	위상 정렬 / 최단 경로 / 조상 탐색
— sorting.py	직접 구현한 병합 정렬
— index.py	역색인
— errors.py	표준 에러 예외 클래스

INIT/BRANCH/SWITCH/COMMIT(저장소·브랜치 관리) + LOG/LOG --sort-by(로그) + PATH/ANCESTORS(탐색) + SEARCH(검색), 총 8개 명령이 모두 동작합니다.

초보자가 코드를 읽을 때는 실행 시작점부터 따라가면 됩니다. cli는 입력 문장을 해석하고 repository 메서드를 호출합니다. repository는 상태를 바꾸거나 알고리즘에 조회를 부탁드립니다. 이를 함수와 클래스로 나누는 것은 과제의 필수 제약이기도 합니다. 테스트는 화면을 거치지 않고 알고리즘을 직접 호출할 수 있습니다.

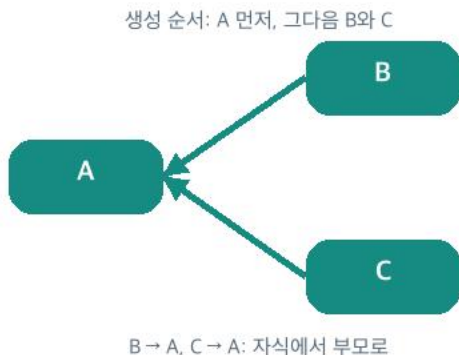
입출력, 저장소 상태, 알고리즘을 나누면 따로 검증할 수 있습니다.

근거: main.py · mini_git/cli.py · repository.py · 평가질문_설명자료.md

커밋 그래프는 왜 DAG인가요?

핵심 · 그림 · 실행

한눈에 이해하기



DAG란?

Directed Acyclic Graph 방향성 비순환 그래프 화살표 방향을 따라 돌아오는 고리가 없음

왜 방향성인가요? — Directed

노드 = 커밋, 간선 = 부모 참조 자식 → 부모 방향으로 연결 B → A는 B의 부모가 A라는 뜻

왜 비순환인가요? — Acyclic

새 커밋은 먼저 만들어진 부모만 참조 기존 커밋의 부모 참조는 유지 계속 이전으로 가므로 출발점에 못 돌아옴

상세 설명 · 코드 · 문답 전체

아주 쉬운 설명: 커밋 하나는 commit.py의 Commit 클래스로 표현됩니다.

```
class Commit:
    def __init__(self, commit_hash, message, author, timestamp, parents
, branch):
        self.hash = commit_hash
        self.message = message
        self.author = author
        self.timestamp = timestamp
        self.parents = list(parents) # 0개, 1개, 또는 2개(병합) 부모 hash
        self.branch = branch
```

각 커밋은 parents로 "자신보다 먼저 존재했던" 커밋(들)만 가리킵니다. 이 참조들을 전부 모으면, 커밋을 노드로 하고 "자식 → 부모" 참조를 간선으로 하는 그래프가 만들어집니다.

왜 방향성(Directed)인가: parents 참조는 항상 "과거"만 가리킵니다. 커밋 B가 커밋 A를 부모로 가진다는 것은 "B가 A보다 나중에 만들어졌다"는 뜻이고, 이 관계는 절대 뒤집힐 수 없습니다(A가 B의 부모이면서 동시에 B가 A의 부모일 수 없음). 그래서 간선에 방향이 있습니다.

왜 비순환(Acyclic)인가: 순환이 있으려면 "A → B → C → A"처럼 자기 자신에게 돌아오는 경로가 있어야 하는데, 이는 "A가 자기 자신보다 나중에 만들어졌다"는 모순을 뜻합니다. COMMIT 명령이 항상 "현재 시점의 HEAD를 부모로 하는 새 커밋"만 만들 수 있게 되어 있어서(repository.commit()), 구조적으로 이미 존재하는 커밋을 미래의 부모로 되돌릴 방법이 없습니다.

방향이 있으면서 사이클이 없는 그래프 = DAG(Directed Acyclic Graph)이고, 병합 커밋으로 부모가 2개가 되어도(브랜치 두 개가 합쳐져도) 이 두 성질은 그대로 유지됩니다.

Q1. 병합 커밋(부모 2개)이 생기면 DAG 성질이 깨지지 않는가?

깨지지 않습니다. 부모가 1개든 2개든, "자식이 부모보다 나중에 만들어졌다"는 방향성과 "자기 자신으로 돌아오는 경로가 없다"는 비순환성은 그대로 유지됩니다. traversal.py의 세 함수(위상 정렬/최단 경로/조상) 모두 commit.parents의 길이를 가정하지 않고 일반적으로 순회하도록 작성해서, 병합 커밋이 있어도 코드 수정 없이 그대로 동작합니다(단위 테스트에서 다이아몬드형 그래프로 이미 검증).

평가 답변: DAG는 Directed Acyclic Graph의 약자로, 방향성 비순환 그래프입니다. 커밋 하나가 노드이고, 자식 커밋에서 부모 커밋으로 향하는 참조가 간선이므로 방향성이 있습니다. 새 커밋은 이미 만들어진 커밋만 부모로 참조하며, 기존 커밋의 부모 참조는 바꾸지 않습니다. 따라서 부모를 한 번 따라갈 때마다 생성 순서가 반드시 더 이전으로 갑니다. 더 이전으로만 이동해서는 출발 커밋으로 돌아올 수 없으므로 순환이 없습니다. 이 두 조건을 만족하여 커밋 그래프는 DAG입니다. 그림 설명: A를 먼저 만들고, A를 부모로 하는 B와 C를 만듭니다. B → A와 C → A는 가능하지만 A가 나중에 생긴 B나 C를 부모로 참조하지는 않습니다. B → A → B처럼 화살표를 따라 출발점으로 돌아오는 경로가 있으면 순환이며 DAG가 아닙니다. 주의: 여기서 이전은 timestamp 값이 아니라 생성 순서를 뜻합니다. PC 시계가 바뀌어도 부모는 생성 당시 존재한 커밋입니다. 병합으로 두 경로가 합쳐져도, 새 병합 커밋이 기존 커밋들을 부모로 참조하고 기존 부모 참조가 유지되면 방향성 순환은 생기지 않습니다. DAG는 반드시 트리일 필요가 없습니다. Mini Git의 정상 COMMIT 명령은 기존 HEAD를 부모로 저장하며 기존 커밋의 parents를 수정하지 않습니다. Commit 클래스 자체는 변경 가능한 Python 객체이므로 외부 코드가 parents를 임의로 변경하는 상황까지 이 생성 규칙이 보장하는 것은 아닙니다. CLI MERGE는 구현하지 않았고, 다중 부모는 자료구조와 테스트에서 지원하지 않습니다.

부모를 따라갈수록 생성 순서가 더 이전으로 가므로, 출발 커밋으로 돌아오는 순환이 없습니다.

근거: mini_git/commit.py · repository.py: commit() · 평가질문_설명자료.md

브랜치와 HEAD는 '끝을 가리키는 이름표'

한눈에 이해하기



현재 브랜치: feature

A에서 feature 생성

main → A

feature → A

현재 브랜치: main

feature에서 B 커밋

main → A

feature → B

B.parents = [A.hash]

실행 확인 · 명령은 한 줄씩 입력

mini-git> · 5페이지 실습과 별도로 처음부터 실행

INIT "Alice Kim" → COMMIT "Initial commit" → BRANCH feature → SWITCH feature → COMMIT "Add login feature" → SWITCH main → LOG

확인: INIT은 기존 데이터를 초기화함. main에는 최초 커밋, feature에는 로그인 커밋

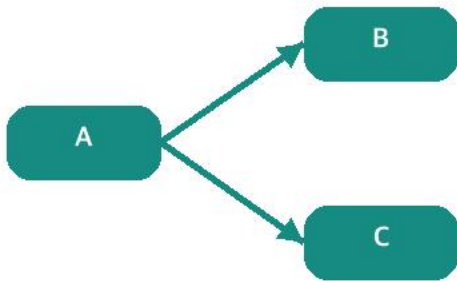
HEAD를 커밋 하나를 담는 별도 객체라고 생각할 필요는 없습니다. 이 구현에서는 current_branch가 현재 브랜치를 나타내고 branches[current_branch]가 현재 커밋입니다. branch는 저장된 커밋을 복제하지 않습니다. 같은 커밋을 가리키는 이름표를 추가합니다. COMMIT은 현재 브랜치만 앞으로 이동시킵니다.

BRANCH는 이름표를 만들고, SWITCH는 사용할 이름표를 고릅니다.

LOG: 선수과목을 먼저 출력하는 방식

핵심 · 그림 · 실행

한눈에 이해하기



A를 먼저 출력 → B와 C가 후보

Kahn 알고리즘의 세 단계

1. 남은 부모 수를 센다: A=0, B=1, C=1 2. A를 출력하고 B·C의 수를 0으로 줄인다
3. 큐에 들어온 B·C를 차례로 출력한다

출력 예

A → B → C A는 반드시 두 자식보다 먼저 B·C 사이의 순서는 FIFO 큐

방향 구분

저장 참조: 자식 → 부모 LOG 계산: 부모 → 자식으로 뒤집어 처리

실행 확인 · mini-git > 5페이지 실습 후

LOG

확인: Initial이 Login·Payment보다 먼저 출력

상세 설명 · 코드 · 문답 전체

traversal.topological_order는 Kahn의 알고리즘을 씁니다. 커밋의 부모 참조(자식→부모)를 뒤집어 부모→자식 간선으로 처리 하므로 부모 수가 이 그래프의 진입차수가 됩니다.

```

in_degree = {h: len(c.parents) for h, c in commits.items()}
children = {h: [] for h in commits}
for h, c in commits.items():
    for p in c.parents:
        children[p].append(h)
  
```

```

queue = [h for h in commits if in_degree[h] == 0] # 부모 없는 커밋부터 시작
result = []
head = 0
while head < len(queue):
    current = queue[head]
    head += 1
    result.append(current)
    for child in children[current]:
  
```

```

        in_degree[child] -= 1
        if in_degree[child] == 0:
            queue.append(child)
  
```

아주 쉬운 설명: "진입차수"는 "이 커밋을 출력하기 전에 먼저 출력돼야 하는 다른 커밋이 몇 개 남았는가"입니다(= 아직 안 나온 부모의 수). 부모가 0개인 커밋(뿌리)부터 출력 후보에 넣고, 하나를 출력할 때마다 그 커밋을 부모로 둔 자식들의 "남은 부모 수"를 하나씩 지웁니다. 0이 되는 순간 그 자식도 이제 출력해도 되는 상태가 되어 후보에 추가됩니다. 이 과정을 반복하면 "부모가

항상 자식보다 먼저 나오는" 순서가 자연스럽게 만들어집니다.

단순 정렬로는 안 되는 이유: "최신순"이라면 timestamp로 정렬하면 그만이지만, "부모가 먼저"라는 조건은 timestamp 하나만으로는 보장되지 않습니다(이론적으로 시계가 부정확하거나 동시에 커밋되면 꼬일 수 있음). 반면 위상 정렬은 애초에 그래프의 부모-자식 "관계"만 보고 순서를 결정하므로, 시각 정보와 무관하게 항상 정확합니다.

실제 확인: 제출/results/02_LOG_위상정렬.txt에서 뿌리 커밋(Initial commit)이 항상 맨 위에 오고, 그 뒤로 두 브랜치(feature/ main)에서 갈라진 자식 커밋들이 나오는 것을 확인했습니다. 단위 테스트에서는 병합 커밋까지 있는 다이아몬드형 그래프로 "부모가 항상 자식보다 앞선 위치"라는 성질을 모든 간선에 대해 검증했습니다(tests/test_mini_git.py).

Q1. 위상 정렬에서 동시에 진입차수가 0이 되는 커밋이 여러 개면 어떤 순서로 나오는가?

FIFO 큐(리스트를 인덱스로 순회)를 쓰기 때문에, "먼저 큐에 들어온" 순서가 그대로 유지됩니다. 이 프로젝트의 전형적인 시나리오(하나의 뿌리 커밋에서 여러 브랜치가 갈라지는 경우)에서는 이 순서가 대체로 "먼저 생성된 순서"와 일치합니다. 과제 명세도 "학습 목적상 부모가 자식보다 먼저"라는 조건만 요구할 뿐 동물 처리 규칙은 정하지 않았으므로, 이 방식은 결정적이고 설명 가능한 선택입니다.

진입차수라는 어려운 용어는 아직 처리되지 않은 선수과목 수라고 풀어 설명합니다. FIFO는 먼저 들어온 것이 먼저 나오는 줄입니다. 동시에 후보가 된 커밋은 이 큐 순서를 따릅니다. 최신순 정렬은 요구사항을 만족하지 않습니다. 위상 정렬은 날짜가 아니라 관계를 사용하므로 부모 우선순위를 보장합니다. 시간 복잡도는 V개 커밋과 E개 연결을 처리하는 $O(V+E)$ 입니다.

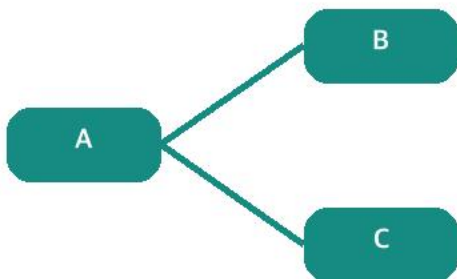
아직 출력하지 않은 부모가 0개인 커밋부터 꺼냅니다.

근거: mini_git/traversal.py: topological_order() · 평가질문_설명자료.md

PATH: 한 칸씩 넓혀 가면 가장 짧은 길

핵심 · 그림 · 실행

한눈에 이해하기



B ↔ A ↔ C: 간선 2개

BFS = 가까운 곳부터

출발지: 거리 0 바로 이웃: 거리 1 그 이웃의 이웃: 거리 2

이 과제의 연결 규칙

부모·자식 연결은 양방향 Login ↔ Initial ↔ Payment Login → Payment의 거리 = 2

경로가 없다면

두 커밋은 있지만 연결 안 됨: No path hash 자체가 없음: Unknown commit: <hash>

실행 확인 · mini-git > 5페이지 실습 후

PATH 5fa901 837f82

PATH 5fa901 5fa901

확인: 현재 LOG의 hash로 교체. 첫 경로는 간선 2개, 자기 자신은 간선 0개

상세 설명 · 코드 · 문답 전체

과제 명세는 "커밋-부모 연결을 무방향 간선으로 간주"하라고 명시합니다. 즉 부모 방향만이 아니라 자식 방향으로도 이동할 수 있는 별도 그래프를 만들어야 합니다.

```

def _undirected_adjacency(commits):
    adjacency = {h: [] for h in commits}
    for h, c in commits.items():
        for p in c.parents:
            adjacency[h].append(p)
            adjacency[p].append(h) # 양방향 간선
    return adjacency
  
```

최단 거리 자체는 이 무방향 그래프에서 BFS 한 번($O(V+E)$)이면 구해집니다. 문제는 "최단 경로가 여러 개 일 때 사전순으로 가장 작은 것"을 골라야 한다는 점입니다. 이를 위해 시작점(start_hash)과 끝점(end_hash) 양쪽에서 각각 BFS 거리를 구해둔 뒤, 시작점부터 한 걸음씩 전진하면서 "이 지점까지의 최단 거리 + 여기서 목적지까지 남은 최단 거리 = 전체 최단 거리"를 만족하는 이웃들(=반드시 어떤 최단 경로 위에 있는 이웃) 중 hash 문자열이 가장 작은 것을 매번 고릅니다.

BFS는 Breadth First Search, 너비 우선 탐색입니다. 물결이 가까운 곳부터 퍼진다고 생각하면 됩니다. 간선마다 비용이 같으므로 먼저 발견한 단계가 최단 거리입니다. PATH는 Git 부모 참조를 그대로 한 방향으로 따라가는 기능이 아니라 과제가 정한 무방향 경로입니다. 같은 출발지와 목적지는 거리 0입니다.

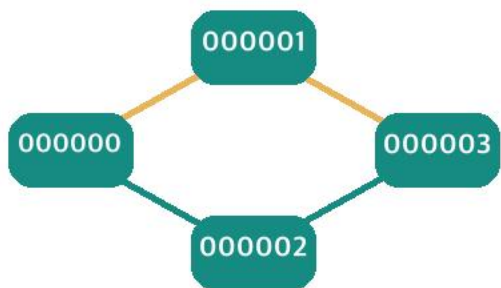
모든 연결의 비용이 1일 때 BFS는 간선 수가 가장 적은 거리를 찾습니다.

근거: mini_git/traversal.py: _bfs_distances(), shortest_path() · 평가질문_설명자료.md

최단 경로가 둘이면 hash 사전순으로

핵심 · 그림 · 실행

한눈에 이해하기



테스트용 다중 부모 그래프

같은 거리의 두 경로

000000 → 000001 → 000003 000000 → 000002 → 000003 두 경로 모두 간선 2개

선택

000001 < 000002 첫 경로의 연결 문자열이 사전순 최소

구현 방법

출발·도착에서 각각 BFS 남은 거리가 1 줄어드는 후보만 선택 그중 hash가 가장 작은 다음 커밋으로

실행 확인 · 일반 터미널 · b3-2 풀더

```
python3 -B -m unittest discover -s tests -v -k merge_ancestors
```

확인: CLI에 MERGE는 없음. 테스트용 다중 부모 그래프에서 사전순 최단 경로 확인 · 마지막 OK 확인

상세 설명 · 코드 · 문답 전체

```
while current != end_hash:
    best_next = None
    for neighbor in adjacency[current]:
        on_shortest_path = (
            dist_from_start.get(neighbor) == dist_from_start[current] + 1
            and dist_from_end.get(neighbor) == dist_from_end[current] - 1
        )
        if on_shortest_path and (best_next is None or neighbor < best_next):
            best_next = neighbor
    current = best_next
    path.append(current)
```

왜 이 방법이 맞는가: 경로를 문자열로 이어붙였을 때 "사전순으로 가장 작다"는 것은, 첫 글자(첫 커밋)부터 비교해서 더 작은 쪽 이 이기고, 같으면 다음 글자(다음 커밋)를 비교하는 방식입니다. 그래서 "지금까지는 같은데, 바로 다음 한 걸음에서 더 작은 쪽을 고른다"를 매 단계 반복하면, 결과적으로 전체 문자열이 사전순으로 가장 작은 경로가 완성됩니다(단, 매 걸음에서 선택지가 "실제로 최단 경로를 유지하는" 이웃으로 한정되어 있어야 하며, 그 조건이 바로 위 on_shortest_path 체크입니다).

실제 확인: 단위 테스트에서 root가 b1→c1과 b2→c2 두 브랜치를 거쳐 merge(병합 커밋)로 합쳐지는 그래프를 만들어, 두 경로 모두 길이가 3으로 동일할 때 사전순으로 더 작은 root→b1→c1→merge가 정확히 선택됨을 확인했습니다(tests/test_mini_git.py). REPL에서도 제출/results/04_PATH_최단경로.txt에서 직접 연결(길이 1)과 공통 부모를 거치는 간접 연결(길이 2)을 모두 확인했습니다.

Q2. PATH의 사전순 타이브레이크 로직을 $O(n^2)$ 이 아니라 왜 더 효율적으로 만들 수 있었는가?

모든 최단 경로를 일일이 나열해서(경로 개수가 그래프 구조에 따라 지수적으로 늘어날 수 있음) 문자열 비교로 최솟값을 고르는 방식도 가능한 하지만, 이 구현은 그 대신 "매 걸음마다 가능한 다음 후보 중 최솟값만 탐욕적으로 선택"하는 방식을 씁니다. 시작 점/끝점 양방향 BFS($O(V+E)$)를 한 번씩 미리 해두면, 이후 경로 재구성은 경로 길이만큼의 반복 × 각 지점의 이웃 수 정도로 끝 나 훨씬 효율적입니다.

이 그림은 부모가 둘인 병합 노드를 포함한 테스트용 다이아몬드 그래프입니다. CLI에서 merge를 실행해서 만든 것이 아닙니다. dStart[next]=dStart[current]+1과 dEnd[next]=dEnd[current]-1 조건을 확인합니다. 작은 hash만 보고 아무 길이나 선택하면 더 긴 경로로 빠질 수 있으므로 최단 조건이 먼저입니다. 모든 최단 경로를 열거하지 않고 두 BFS와 경로 복원으로 계산합니다.

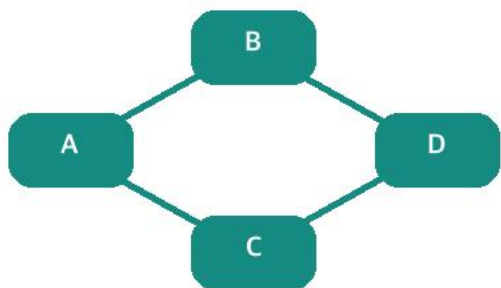
최단 거리 조건을 지키는 후보 중 다음 hash가 작은 쪽을 고릅니다.

근거: tests/test_mini_git.py: test_merge_ancestors_and_lexicographic_path · 평가질문_설명자료.md

ANCESTORS: 부모의 부모를 빠짐없이 찾기

핵심 · 그림 · 실행

한눈에 이해하기



테스트용 다중 부모 그래프

예: D의 부모가 B·C

B의 부모: A C의 부모: A D의 조상: B, C, A A는 한 번만, D는 제외

visited = 방문 기록

set으로 이미 방문한 hash를 확인 중복으로 만난 조상은 건너뛰며 큐는 앞을 삭제하지 않고 인덱스로 읽음

복잡도

$O(V+E)$ V: 도달 가능한 조상 수 E: 살펴보는 부모 연결 수

실행 확인 · mini-git > 5페이지 실습 후

ANCESTORS 837f82

ANCESTORS 0a7990

확인: 현재 LOG의 hash로 교체. Payment의 조상은 Initial, Initial은 No ancestors.

상세 설명 · 코드 · 문답 전체

```

def ancestors(commits, commit_hash):
    visited = set()
    queue = list(commits[commit_hash].parents)
    result = []
    head = 0
    while head < len(queue):
        h = queue[head]
        head += 1
        if h in visited:
            continue
        visited.add(h)
        result.append(h)
        queue.extend(commits[h].parents)
    return result
  
```

아주 쉬운 설명: "부모의 부모의 부모..."를 계속 따라 올라가는 탐색입니다. 부모 목록에서 하나씩 꺼내 방문 처리하고, 그 커밋의 부모도 다시 큐에 넣는 식으로 반복합니다. 병합 커밋 때문에 같은 조상에 여러 경로로 도달할 수 있어(예: 다이아몬드형 구조), visited 집합으로 이미 본 커밋은 건너뛰어 중복과 무한 루프를 막습니다. 큐는 앞의 원소를 삭제하지 않고 인덱스로 읽으므로 $O(V+E)$ 로 탐색합니다.

가족의 조상 목록을 만드는 것과 비슷하지만 여러 부모 경로가 같은 조상에 합쳐질 수 있습니다. set은 이미 본 hash를 빠르게 확인하는 집합입니다. 초기 큐에는 조회 대상의 부모만 넣으므로 자신은 결과에 들어가지 않습니다. 이전의 list.pop(0)은 원소들을 당겨야 했고, 지금은 head 인덱스를 이동하여 큐 비용을 줄였습니다.

부모 방향만 탐색하고, 이미 본 조상은 한 번만 포함합니다.

근거: mini_git/traversal.py: ancestors() · 평가질문_설명자료.md

정렬은 병합 정렬을 직접 구현했습니다

핵심 · 그림 · 실행

한눈에 이해하기

```
[4, 1, 3, 2]

[4, 1] [3, 2]

[4] [1] [3] [2]

[1, 4] [2, 3]

[1, 2, 3, 4]
```

나누기

```
[4, 1, 3, 2] [4, 1] [3, 2] [4] [1] [3] [2]
```

합치기

```
[1, 4] [2, 3] [1, 2, 3, 4]
```

평가 질문: 왜 퀵 정렬이 아닌가요?

병합 정렬은 평균·최악 $O(n \log n)$ 같은 값의 원래 순서를 지키는 안정 정렬 일반적 퀵 정렬은 최악 $O(n^2)$, 보통 불안정

실행 확인 · mini-git> 5페이지 실습 후

```
LOG --sort-by=date
LOG --sort-by=author
```

확인: 날짜·작성자 오름차순. 한 작성자만 있으면 author 정렬의 변화가 보이지 않을 수 있음

상세 설명 · 코드 · 문답 전체

sorting.merge_sort는 병합 정렬을 일반화된 형태(비교 키를 함수로 받음)로 구현했습니다.

```
def merge_sort(items, key):
    if len(items) <= 1:
        return list(items)
    mid = len(items) // 2
    left = merge_sort(items[:mid], key)
    right = merge_sort(items[mid:], key)
    return _merge(left, right, key)

def _merge(left, right, key):
    merged = []
    i = j = 0
    while i < len(left) and j < len(right):
        if key(left[i]) <= key(right[j]): # <= → 안정 정렬의 핵심
            merged.append(left[i]); i += 1
        else:
            merged.append(right[j]); j += 1
    merged.extend(left[i:])
    merged.extend(right[j:])
    return merged
```

표준 sorted와 list.sort를 사용하는 것은 금지되어 있습니다. key 함수는 비교할 값을 꺼내는 도구입니다. date에서는 timestamp, author에서는 author를 꺼냅니다. 일반적인 병합 정렬은 임시 배열 공간 $O(n)$ 을 사용합니다. 퀵 정렬이 언제나 나쁜 것은 아니며, 이 과제에서는 최악 시간과 안정성이 설명하기 쉬운 병합 정렬을 선택했습니다.

반으로 나누고, 작은 값부터 합칩니다. 날짜·작성자는 비교 키만 바꿉니다.

근거: mini_git/sorting.py: merge_sort(), _merge() · 평가질문_설명자료.md

O(n log n)과 안정 정렬을 쉬운 말로

핵심 · 그림 · 실행

작업량

n = 데이터 개수 8개를 나누면 $8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 각 병합 층의 전체 작업은 약 n 평균·최악 시간: $O(n \log n)$

안정 정렬

정렬 전: Bob, Alice①, Alice② 정렬 후: Alice①, Alice②, Bob 키가 같으면 왼쪽을 먼저 채택: $\leq$ ①·②는 설명용 순서 표식

실행 확인 · 일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k sort_keys
```

확인: 정렬 키·안정성·원소 보존·원본 불변 확인 · 마지막 OK 확인

상세 설명 · 코드 · 문답 전체

시간복잡도: 리스트를 절반씩 나누는 분할 과정은 입력이 어떻게 배치돼 있든 항상 정확히 반으로 쪼개지므로, 재귀 깊이가 $\log_2 n$ 입니다. 각 깊이 층에서 이루어지는 병합 작업의 총량은 그 층에 있는 원소 전체 개수인 n입니다. 따라서 전체 시간은 $O(n) \times O(\log n) = O(n \log n)$ 이며, 이는 입력의 초기 배치와 무관하게 항상 성립하므로 평균과 최악이 동일하게 $O(n \log n)$ 입니다. (참고로 퀵 정렬은 평균은 $O(n \log n)$ 이지만, 피벗을 잘못 고르면(예: 이미 정렬된 배열에서 항상 첫 원소를 피벗으로 고르는 구현) 최악의 경우 $O(n^2)$ 까지 나빠질 수 있습니다 — 병합 정렬은 이런 입력 의존성이 없습니다.)

안정 정렬 여부: `_merge`에서 두 값을 비교할 때 $<$ 가 아니라 $\leq$ 를 씁니다. 즉 두 값이 같을 때도 "왼쪽 그룹(원래 앞쪽에 있던 원소들)"을 먼저 채택합니다. 그 결과, 정렬 기준 값이 동일한 원소들끼리는 정렬하기 전의 상대적 순서가 정렬 후에도 그대로 유지 됩니다 — 이것이 안정 정렬(Stable Sort)의 정의입니다.

이게 왜 중요한가: `LOG --sort-by=author`에서 같은 작성자가 커밋을 여러 개 가진 경우, "동률 처리 규칙은 자유"라는 과제 조건 안에서 "원래(위상 정렬) 순서를 유지한다"는 직관적이고 재현 가능한 규칙을 별도 코드 없이 안정 정렬의 성질만으로 얻을 수 있습니다.

실제 확인: 단위 테스트에서 작성자가 Charlie/Alice/Bob/Alice인 가짜 커밋 4개를 만들어 author 키로 정렬한 결과, 두 Alice 커밋 중 원본에서 먼저 나온 것이 정렬 후에도 먼저 나오음을 확인했습니다. 또한 무작위 데이터 1,000개에서 정렬 순서, 원소 보존, 원본 불변, 같은 키의 상대 순서 보존을 확인했습니다(`tests/test_mini_git.py`).

Q3. 병합 정렬 대신 퀵 정렬을 선택했다면 무엇이 달라졌을까?

퀵 정렬은 평균 $O(n \log n)$ 으로 병합 정렬과 같지만, (a) 최악의 경우 $O(n^2)$ 로 나빠질 수 있고 (b) 일반적인 구현은 원소 교환(`swap`) 방식이라 안정 정렬이 아닙니다(같은 값이라도 위치가 바뀔 수 있음). 이 과제는 "동률 처리"가 중요한 `LOG --sort-by`에 정렬 결과를 쓰므로, 안정성이 보장되고 시간복잡도가 입력에 흔들리지 않는 병합 정렬을 선택했습니다.

시간복잡도는 실제 초 단위 실행 시간이 아니라 입력이 커질 때 작업량이 얼마나 늘어나는지 표현합니다. 안정성은 프로그램이 안 죽는다는 뜻이 아니라 같은 비교값끼리 원래 순서를 지킨다는 뜻입니다. $\leq$ 를 쓰면 두 값이 같을 때 원래 앞에 있던 왼쪽 원소를 먼저 꺼냅니다. 테스트는 순서뿐 아니라 원본 목록이 바뀌지 않았는지도 확인합니다.

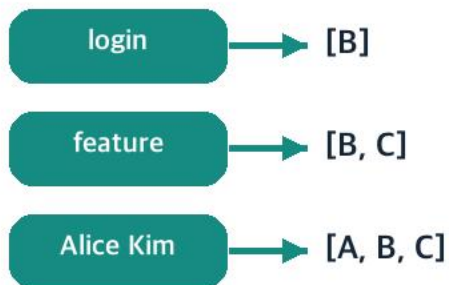
한 층에서 n개를 다루고 층은 $\log_2 n$ 개. 같은 키는 원래 순서를 보존합니다.

근거: `mini_git/sorting.py` · `tests/test_mini_git.py: test_sort_keys_and_stability` · 평가질문_설명자료.md

역색인: 책 뒤의 '찾아보기'

핵심 · 그림 · 실행

한눈에 이해하기



메시지를 토큰으로

Add Login Feature → add, login, feature 공백 split + 소문자 lower

색인 예

login → [B] feature → [B, C] Alice Kim → [A, B, C]

검색 비용

전체 순회: 커밋 수 N에 비례 단어·작성자 색인 조회: 평균 O(1) 결과 k개를 복사·출력: O(k)

실행 확인 · mini-git > 5페이지 실습 후

```

SEARCH login
SEARCH feature
SEARCH --author="Alice Kim"
  
```

확인: 5페이지의 새 실습 기준: 각각 1개, 2개, 3개

상세 설명 · 코드 · 문답 전체

index.py의 InvertedIndex는 커밋이 만들어질 때마다(COMMIT 실행 시) 메시지를 미리 분해해 색인해 둡니다.

```

def add_commit(self, commit):
    tokens = tuple(token.lower() for token in commit.message.split())
    self._message_tokens[commit.hash] = tokens
    for keyword in dict.fromkeys(tokens): # 한 커밋 안의 반복 단어 제거
        self._keyword_index.setdefault(keyword, []).append(commit.hash)
    self._author_index.setdefault(commit.author, []).append(commit.hash)
  
```

아주 쉬운 설명: 책 뒤에 붙어있는 "찾아보기(색인)"와 같은 원리입니다. 책을 읽을 때마다 처음부터 그 단어를 찾는 대신, 미리 "이 단어는 몇 페이지에 나온다"를 정리해 둔 목록을 보고 바로 그 페이지로 넘어가는 것과 같습니다. 여기서는 "이 단어는 어떤 커밋(들)에 나온다"를 미리 정리해 둡니다.

시간복잡도 비교:

| 방식 | SEARCH 처리 방법 | 시간복잡도 | |---|---|---| 순회 검색(역색인 없음) | 커밋 N개를 전부 훑으며 메시지 안에 키워드가 있는지 매번 검사 | O(N) | | 단어 역색인 검색 | keyword_index[키워드]를 딕셔너리에서 바로 조회 | 평균 O(1) + 결과 복사 O(k), 검색어 정규화 비용 별도 |

커밋이 10개면 차이가 크게 안 느껴지지만, 커밋이 100만 개라면 순회 검색은 매번 100만 번을 다 훑어야 하는 반면, 역색인은 여전히 거의 즉시 결과를 돌려줍니다. 대신 "커밋을 만들 때마다 미리 색인해두는" 비용(메시지 길이에 비례하는 O(m))을 매번 지 불하는 트레이드오프입니다 — 검색이 저장보다 훨씬 자주 일어나는 상황(Git 로그처럼 한 번 만든 커밋을 여러 번 찾아보는 경우)에 유리한 구조입니다.

author 검색도 같은 원리: author_index[작성자명]으로 바로 조회하므로, "이 사람이 만든 커밋을 모두 보여줘"도 전체 순회 없이 평균 O(1) 조회와 O(k) 결과 복사로 처리됩니다. 이름 길이에 따른 해싱 비용은 별도입니다.

실제 확인: 제출/results/06_SEARCH.txt에서 search login이 정확히 1건(Add login feature)을, search --author="Alice Kim"이 3건(전체)을, 없는 키워드는 0건을 반환함을 확인했습니다. 대소문자가 섞인 메시지도 "Login"으로 검색하면 매칭되는지(소문자 정규화 확인) 별도 단위 테스트로도 검증했습니다(tests/test_mini_git.py).

역색인의 역은 커밋에서 단어를 찾는 대신 단어에서 커밋을 찾는다는 의미입니다. 키워드를 담는 dict와 작성자를 담는 dict 두 종류를 사용합니다. 한 메시지의 login이 세 번 반복되어도 그 커밋 hash는 해당 목록에 한 번만 들어갑니다. 쓰기 시점의 색인 비용과 추가 메모리 대신 검색 속도를 얻습니다. 해싱과 정규화는 문자열 길이만큼 비용이 별도로 듭니다.

커밋을 만들 때 단어별 커밋 목록을 준비해서, 검색 때 바로 꺼냅니다.

근거: mini_git/index.py: add_commit(), search_keyword(), search_author() · 평가질문_설명자료.md

공백 포함 검색은 '연속 구문'으로

핵심 · 그림 · 실행

SEARCH "login feature"

Add Login Feature → 일치 login another feature → 불일치 feature login → 불일치 대소문자·공백 수는 무시

후보 선택과 비용

login 목록과 feature 목록 중 작은 목록 선택 후보의 토큰이 연속해서 같은지 확인 구문 길이 q , 후보 토큰 합 T 후보 검사: 최악 $O(T \cdot q)$

실행 확인 · mini-git> · 5페이지 실습 이후 추가 입력

COMMIT "login another feature"

COMMIT "feature login"

SEARCH "login feature"

확인: Add login feature만 일치. 추가 두 커밋은 단어가 연속하지 않거나 순서가 반대여서 제외

상세 설명 · 코드 · 문답 전체

공백 포함 구문 검색: SEARCH "login feature"는 가장 드문 단어의 역색인 후보만 가져와 메시지 토큰이 연속해서 일치하는지 검사합니다. 토큰의 순서와 반복 횟수는 구분하고 대소문자와 공백 수는 무시합니다. 구문 길이를 q , 후보 메시지들의 토큰 수 합을 T 라 하면 후보 검사는 최악 $O(T \cdot q)$ 이며, 결과 출력 $O(k)$ 와 검색어 정규화 비용이 추가됩니다. 후보가 전체 커밋에 해당할 수 있으므로 구문 검색을 항상 $O(1)$ 이라 말할 수는 없습니다.

이 검색은 부분 문자열 검색이 아닙니다. login으로 logging을 찾지 않습니다. 여러 단어를 검색할 때 단어가 다 있다는 사실만으로는 부족합니다. 붙어 있는지, 순서가 맞는지, 같은 단어를 반복한 구문인지 확인해야 합니다. 모든 커밋에 흔한 단어라면 후보가 많아질 수 있으므로 모든 검색이 $O(1)$ 이라고 말하지 않습니다.

드문 단어의 후보만 확인하므로 전체 저장소를 훑지 않습니다.

근거: mini_git/index.py: search_keyword() · 제출/results/10_보완_회귀검증.txt · 평가질문_설명자료.md

평가 질문: dict와 hash를 왜 썼나요?

핵심 · 그림 · 실행

Q. list 대신 dict인 이유?

list: 같은 hash를 처음부터 찾아야 함 → $O(N)$ dict: hash를 키로 평균 $O(1)$ 조회 Repository.commits[hash]로 바로 찾을 수 있음

Q. hash 중복은 어떻게 막나요?

내용 + 시퀀스를 SHA-1으로 해싱 앞 6자리를 기존 hash와 비교 충돌이면 새 시퀀스로 다시 시도 카운터만으로 충돌이 사라지지는 않음

실행 확인 · 일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k hash_collision
```

확인: hash 충돌을 강제로 발생시켜 재시도와 식별자 중복 방지 확인 · 마지막 OK 확인

상세 설명 · 코드 · 문답 전체

Q2. 커밋 저장소로 왜 dict를 썼는가? list로도 되지 않는가?

PATH/ANCESTORS/COMMIT(부모 조회) 등 대부분의 연산이 "이 hash를 가진 커밋을 찾아라"라는 형태입니다. list에서 특정 값을 찾으려면 처음부터 끝까지 훑어야 해서 $O(n)$ 이지만, dict(해시맵)는 평균 $O(1)$ 에 바로 찾을 수 있습니다. 과제 명세도 "커밋 저장소는 커밋 hash로 빠르게 찾을 수 있어야 한다(예: 해시맵 기반)"고 명시하고 있고, 이 과제는 dict 사용을 허용하므로 (b3-1의 Mini Redis와 달리 dict/list/set은 자유롭게 사용 가능) 표준 dict를 그대로 활용했습니다.

Q3. 커밋 hash 중복은 어떻게 막았는가?

commit.py의 generate_commit_hash는 메시지/작성자/시각/부모 목록에 더해 세션 동안 계속 증가하기만 하는 카운터를 함께 SHA-1으로 해싱합니다. 같은 내용도 시퀀스 번호를 바꾸어 새로운 입력으로 해싱합니다. 서로 다른 입력도 같은 출력 hash를 만들 수 있으므로, 유일성을 보장하는 핵심은 기존 hash와 비교하고 충돌하면 재시도하는 검사입니다. 6자리 공간은 166개로 유한합니다. 충돌을 강제로 발생시키는 회귀 테스트로 재시도 동작을 확인했습니다.

hash는 이 구현에서 짧은 문자열 식별자입니다. 같은 내용을 다시 커밋해도 시퀀스가 바뀌어 새로운 입력으로 해싱합니다. 서로 다른 입력의 출력이 같을 수 있으므로 존재 여부 검사와 재시도가 핵심입니다. 6자리 공간은 16의 6승으로 유한하다는 한계도 있습니다. 강제로 같은 digest가 나오게 만든 테스트로 재시도를 확인했습니다.

dict는 빠른 조회를 맡고, hash 중복 검사는 식별자의 유일성을 지킵니다.

근거: mini_git/commit.py · repository.py · test_hash_collision_retry · 평가질문_설명자료.md

평가 질문: CLI 정책을 설명해 보세요

핵심 · 그림 · 실행

질문 / 실제 동작과 이유

LOG의 [branch]는? / 생성 당시 브랜치 표시, 현재 HEAD 표시는 아님 정렬 LOG에도 표시? / 필수 필드 외에 같은 출력 형식을 유지 INIT을 다시 하면? / 커밋·브랜치·역색인을 초기화 기존 이름으로 BRANCH? / Invalid args, 기존 HEAD 보존 INIT 전에 명령 입력? / Run INIT first 안내 대소문자·공백 인자? / 명령 upper, 문자열 shlex로 따옴표 파싱

실행 확인 · mini-git> · 5페이지 실습 후

```
BRANCH feature
SWITCH missing
COMMIT "    "
```

확인: 기존 feature가 있으면 Invalid args / Unknown branch: missing / Invalid args

상세 설명 · 코드 · 문답 전체

Q1. LOG의 [branch] 표시는 과제 예시와 다르게 --sort-by 결과에도 항상 나오게 했는데, 왜인가?

과제 예시는 --sort-by 결과에서 [branch] 태그를 생략했지만, "출력 포맷은 자유이나 hash/author/timestamp/message가 식별 가능해야 한다"고만 명시되어 있어 형식 선택의 여지가 있었습니다. 정보가 더 많은 쪽이 유용하다고 판단해 두 출력 모두 일관되게 [branch]를 포함시켰습니다.

Q2. INIT을 다시 호출하면 기존 데이터는 어떻게 되는가?

저장소 전체(커밋/브랜치/역색인)를 새로 초기화합니다. 과제 명세가 "INIT은 저장소 초기화"라고 정의하고 있고, 별도의 "재초기화 방지" 요구사항이 없었기 때문에 매번 새로 시작하는 것이 가장 직관적이라고 판단했습니다.

Q3. BRANCH를 이미 존재하는 이름으로 다시 호출하면?

Invalid args로 거부하고 기존 브랜치의 HEAD를 보존합니다. BRANCH는 새 이름표를 추가하는 기능이므로

기존 이름표를 조용히 이동시키지 않습니다. 다른 브랜치에서 BRANCH main을 입력해도 모든 브랜치 포인터와 현재 브랜치가 유지되는지 회귀 테스트로 확인했습니다.

Q4. INIT 이전에 다른 명령을 입력하면 어떻게 되는가?

Repository not initialized. Run INIT first.라는 안내 메시지를 반환합니다. 이는 과제 명세의 최소 3종 표준 에러에는 없지만, "잘못된 입력에 대한 최소 에러 메시지를 표준화한다"는 조항이 그 3개가 전부라고 못박지는 않았기 때문에, 사용성을 위해 추가한 확장입니다.

기존 설명 문서에는 BRANCH 덮어쓰기 정책이 남아 있었지만 현재 코드는 중복 이름을 거부합니다. 발표에서는 현재 구현을 기준으로 답합니다. [branch]는 커밋의 생성 당시 메타데이터라 지금 브랜치가 가리키는 곳이라는 의미가 아닙니다. 표준 오류에는 Invalid args, Unknown branch, Unknown commit이 있고, 초기화 전 안내는 별도 확장입니다. 재초기화는 저장 데이터를 없애므로 데모를 다시 시작할 때 사용합니다.

상태를 바꾸는 정책과 출력용 정보를 구분해서 답합니다.

근거: mini_git/cli.py · repository.py · tests/test_mini_git.py · 평가질문_설명자료.md

평가 질문: 과제 제약을 지켰나요?

핵심 · 그림 · 실행

사용한 것

dict: 커밋·브랜치·색인 list: 부모·큐·결과 set: 조상 방문 기록 표준 라이브러리만 사용

사용하지 않은 것

sorted(), list.sort() 그래프 전용 라이브러리 네트워크 통신·파일 내용 추적

선택 과제

Diff·Merge 명령 미구현 복수 정렬 성능 비교 미구현 메모리만 유지, 영속성은 선택

실행 확인 · 일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v -k no_standard_sort
```

확인: 표준 정렬 호출·그래프 전용 라이브러리 사용 여부 검사 · 마지막 OK 확인

상세 설명 · 코드 · 문답 전체

Q1. sorted()/list.sort()를 정말 하나도 안 썼는가?

파이썬에는 sorted()와 list.sort()라는 기본 정렬 기능이 있습니다. 이 과제는 정렬 원리를 직접 구현하는 연습이므로, 기본 기능 대신 직접 만든 merge_sort()를 사용했습니다. 코드에서 sorted()나 .sort()를 실제로 호출했는지 검사했습니다. 주석에 이름만 적힌 것은 사용한 것이 아닙니다. AST 검사는 코드를 문법 구조로 분석해 실제 함수 호출을 확인하는 방법입니다.

정렬 결과도 확인했습니다.

- 오름차순: 작은 값부터 나오는가?
- 원소 보존: 데이터가 빠지거나 추가되지 않았는가?
- 같은 값의 순서 유지: Alice①이 Alice②보다 앞에 남는가?
- 원본 불변: 입력 데이터는 그대로인가?

발표 답변: 기본 정렬 함수 대신 병합 정렬을 직접 구현했습니다. 금지된 함수 사용 여부와 정렬 결과를 테스트했습니다.

Q2. 그래프 전용 라이브러리를 안 썼다는 건 어떻게 확인했는가?

import와 from은 다른 파일이나 도구의 기능을 가져오는 명령입니다. 이 명령들을 살펴보면 어떤 도구를 사용했는지 알 수 있습니다.

가져온 도구와 용도는 다음과 같습니다.

- hashlib: 커밋 hash 생성
- itertools: 순번 생성
- datetime: 날짜·시간 처리
- shlex: 입력 명령과 따옴표 해석
- mini_git 내부 모듈: 프로젝트에서 직접 작성한 코드

그래프 탐색을 대신 해주는 networkx 같은 라이브러리는 사용하지 않았습니다.

발표 답변: 코드의 import 문을 확인했습니다. 날짜나 hash 생성 도구와 직접 작성한 코드만 사용했고, 그래프 탐색은 직접 구현했습니다.

Q3. dict/list/set을 자유롭게 쓸 수 있다는 걸 어떻게 활용했는가?

쉽게 말하면 "파이썬의 기본 보관함 세 가지를 용도에 맞게 사용했다"는 뜻입니다.

dict: 이름표로 값을 찾는 보관함입니다. hash로 커밋을 찾고, 브랜치와 검색 색인을 관리합니다.

커밋 저장소: "5f8188" → 해당 커밋

브랜치 관리: "main" → 마지막 커밋의 hash

검색 색인: "login" → 해당 커밋들의 hash 목록

list: 여러 값을 순서대로 담는 목록입니다. 부모 커밋 목록, 탐색 대기 목록, 결과 목록에 사용합니다.

set: 같은 값이 중복되지 않는 모음입니다. 이미 방문한 커밋을 기록합니다.

A를 처음 발견 → 방문 기록에 넣기

A를 다시 발견 → 이미 방문했으므로 건너뛰기

따라서 같은 조상 A를 결과에 여러 번 넣지 않습니다. 방문 여부 확인도 평균적으로 빠릅니다.

과제에서 이 기본 자료형은 사용을 허용하므로 활용했습니다. 탐색과 정렬 알고리즘은 직접 구현했습니다.

발표 답변: dict는 커밋과 검색 정보를 빠르게 찾는 데, list는 부모와 결과 목록을 담는 데, set은 중복 방문을 막는 데 사용했습니다.

Q4. 네트워크/파일 추적/영속성을 구현하지 않은 이유는?

과제에서 이 기능들을 만들 필요가 없다고 정했기 때문입니다. 이번 과제는 커밋 사이의 관계와 탐색·정렬·검색 원리를 배우는 데 집중합니다.

• 네트워크 통신: GitHub 같은 다른 서버로 데이터를 보내거나 받는 기능입니다. 이번에는 구현하지 않았습니다.

• 파일 내용 추적: 실제 소스 파일의 내용과 변경 사항을 기록하는 기능입니다. 이번에는 hash·메시지·작성자·시간·부모 관계 같은 커밋 정보만 관리합니다.

• 영속성: 프로그램을 종료해도 데이터가 남도록 파일 등에 저장하는 기능입니다. 이번에는 메모리에만 보관하므로 종료 후 다시 실행하면 커밋과 브랜치가 사라집니다.

발표 답변: 과제 범위에 따라 파일 내용과 네트워크 기능은 생략했고, 저장도 메모리에서만 합니다. 커밋 그래프의 탐색·정렬·검색을 직접 구현하는 데 집중했습니다.

AST는 파이썬 코드를 문법 구조로 읽은 것입니다. 주석에 sorted라는 단어가 있어도 실제 호출인지 구분할 수 있습니다. 현재 구현의 호출 AST를 검사하고 import 문도 직접 확인했습니다. 테스트 자체의 무작위 정렬 검증도 표준 정렬 결과에 기대지 않고 오름차순, 원소 보존, 안정성을 확인합니다. 기본 자료형은 허용되므로 dict를 직접 만들 필요는 없습니다.

과제의 목표는 커밋 그래프 알고리즘이며, 파일·네트워크 기능은 범위 밖입니다.

근거: 과제: 제약 사항 · tests/test_mini_git.py: AST 검사 · 평가질문_설명자료.md

평가 질문: 알고리즘을 어떻게 믿을 수 있나요?

핵심 · 그림 · 실행

대상 / 독립적인 확인 방법

LOG / 모든 부모의 출력 위치 < 자식 위치 PATH / 작은 그래프의 모든 단순 경로와 비교 ANCESTORS / 별도 스택 탐색의 조상 집합과 비교 정렬 / 오름차순·원소 보존·같은 키의 순서·원본 불변 검색 / 반복 단어·연속 구문·저장소 순회 금지 실제 CLI / main.py 실행, 프롬프트·exit/quit 확인

실행 확인 · 일반 터미널 · b3-2 폴더

```
python3 -B -m unittest discover -s tests -v
```

확인: Ran 15 tests와 OK 확인. 각 테스트 이름으로 검사 대상 확인

상세 설명 · 코드 · 문답 전체

Q1. 각 알고리즘이 올바르게 동작하는지 어떻게 검증했는가?

REPL을 통한 통합 테스트 외에, sorting.py/traversal.py/index.py를 직접 import해서 모듈 단위로도 검증했습니다. 병합 정렬은 서로 다른 작성자·시각을 가진 가짜 커밋과 무작위 정수 1,000개로, 그래프 탐색은 병합 커밋이 있는 다이아몬드형 구조를 직접 만들어 위상 정렬 순서·최단 경로의 사전순 타이브레이크·조상 집합을 모두 정밀 검증했습니다 (제출/실행결과.md의 "알고리즘 검증" 및 tests/test_mini_git.py 참고).

Q2. 과제의 "8. 결과 예시"와 실제 실행 결과가 일치하는가?

명령의 구조(초기화→커밋→브랜치→전환→커밋→LOG→PATH→SEARCH)와 각 명령의 출력 형식(Initialized repository. / [branch hash] message / commit hash (author, timestamp) [branch] / Path: h1 -> h2 / Found N commit(s):)이 모두 예시와 동일하게 재현됩니다. 실제 hash 값은 SHA-1 기반으로 매 실행마다 새로 생성되므로 예시의 a1b2c3과는 다르지만, 과제 원문도 "실제 문구와 디자인은 달라도 된다"고 명시하고 있어 문제가 되지 않습니다.

막히는 질문이 나오면 README.md, 실제 코드(cli.py → repository.py → traversal.py/sorting.py/index.py 순서), 그리고 제출/실행결과.md와 제출/results/의 실제 실행 결과를 같이 보여주면서 설명하는 것이 가장 확실합니다.

정답 검증용 방식은 제품 알고리즘과 다른 것이 좋습니다. PATH 구현은 BFS를 쓰고, 작은 그래프의 검증은 모든 단순 경로를 열거하여 길이와 문자열을 비교합니다. 이 느린 방식은 작은 테스트에만 사용합니다. 무작위 DAG는 고정 seed를 사용해 다시 같은 입력을 만들 수 있습니다. 과제 예시와는 hash 값이 아니라 흐름과 조건을 비교합니다. 15개 통과가 모든 가능한 입력의 무결함을 증명하는 것은 아닙니다.

구현과 다른 방법으로 기대 결과를 만들고, 같은 결과인지 비교했습니다.

근거: tests/test_mini_git.py · 제출/results/11_자동검증.txt · 평가질문_설명자료.md

이제 내 말로 1분 동안 설명해 보세요

핵심 · 그림 · 실행

발표 순서

① b3-2에서 python3 main.py로 실행했습니다. ② 커밋을 분기하고 LOG·PATH·SEARCH를 확인했습니다. ③ 자동 테스트 15개가 통과했고 기록은 제출/results에 있습니다. ④ LOG는 부모 우선, PATH는 BFS, 조상은 부모 탐색입니다. ⑤ 정렬은 안정 병합 정렬, 검색은 역색인을 직접 구현했습니다.

발표 전 체크

실제 hash로 PATH 입력하기 검증 파일 11과 테스트 소스 열어 두기 Merge 명령과 테스트용 병합 그래프 구분하기

상세 설명 · 코드 · 문답 전체

> "커밋은 부모 hash 목록만 들고 있는 노드이고, 이 참조들이 모여 DAG를 > 이룹니다. LOG는 Kahn의 위상 정렬로 부모가 항상 자식보다 먼저 나오게 > 했고, PATH는 부모-자식 연결을 무방향 간선으로 보고 양쪽에서 BFS 거리를 > 구해 사전순으로 가장 작은 최단 경로를 탐욕적으로 재구성했습니다. > ANCESTORS는 부모 방향 BFS로 도달 가능한 모든 조상을 찾습니다. 정렬은 > sorted() 없이 병합 정렬을 직접 구현해 $O(n \log n)$ 을 항상 보장하고 > 안정 정렬 성질로 동률 처리를 자연스럽게 해결했습니다. 검색은 커밋 > 생성 시점에 미리 만들어두는 역색인으로 후보를 가져옵니다. 단어·작성자 조회는 > 평균 $O(1)$ 에 결과 처리 $O(k)$ 가 추가되고, 구문 검색은 후보 메시지를 확인합니다."

연습 답변: 저는 커밋 메타데이터를 관리하는 Mini Git을 만들었습니다. b3-2에서 실행하고 브랜치를 나누어 부모 관계를 확인했습니다. LOG는 부모가 먼저 나오도록 위상 정렬하고, PATH는 연결을 양방향으로 보아 BFS로 최단 거리를 구합니다. 조상은 부모 방향으로 찾고, 정렬은 병합 정렬, 검색은 역색인으로 구현했습니다. Python 3.12.13에서 테스트 15개가 통과했으며 결과와 코드는 제출/results와 tests 폴더에 있습니다. 질문을 받으면 해당 소스 파일과 결과를 함께 보여 주세요.

실행 명령 하나, 검증 근거 하나, 알고리즘 이유 하나를 꼭 말하세요.

근거: README.md · 최종검수결과.md · 제출/실행결과.md · 평가질문_설명자료.md